

Curated File Guide: server.mjs

The local backend that saves drafts, runs compilers, checks GitHub publishing access, imports/pushes site files, handles updates, and relays AI.

Before The Code: File Overview

Abstract

server.mjs is the local nervous system of the OMB Portfolio Builder. It is not the public website and it is not the Electron shell. It is the private backend that runs on the owner's machine and gives the builder the powers a browser page should not have by itself: reading and writing local files, saving drafts, accepting uploads, running compilers, using Git, checking publishing authorization, importing a target site, launching installers, and relaying AI requests. The file is intentionally broad because the builder is local-first. Instead of depending on a hosted database and a remote application server, the desktop app starts this Node server beside the user-owned workspace and talks to it through local API endpoints.

The most important idea is boundary control. The frontend can draw buttons and collect input, but it cannot safely decide where files are written, which Git repository receives a push, whether a command should run, or whether a request came from the local computer. server.mjs owns those decisions. It is the layer that turns a UI action into a guarded filesystem, compiler, Git, installer, or AI operation.

Introduction

When the builder starts, Electron loads a local page. That page needs catalog data, templates, upload handling, compile actions, publishing actions, and update checks. A static HTML page cannot perform those operations directly without either exposing dangerous privileges or running into browser security walls. server.mjs solves that by becoming a small local application server. It serves the builder files, exposes API endpoints under /api, and keeps privileged work in Node where the filesystem, process runner, and Git commands are available.

A useful mental model is to treat this file like a workshop manager. The frontend is the workbench surface where the user sees projects, editors, buttons, and dialogs. server.mjs is the person behind the counter who knows where the tools are, where files belong, whether a customer has permission to ship something, and whether the requested operation is safe. If the frontend says 'save this PDF under this project,' server.mjs sanitizes the path and writes it under the correct docs folder. If the frontend says 'compile this C++ program,' server.mjs finds g++, creates a temporary run folder, captures terminal output, and returns a result object. If the frontend says 'apply to site,' server.mjs checks Git authorization first, then synchronizes publishable files and pushes them.

Background Information

The file uses built-in Node modules rather than a large web framework. createServer from node:http accepts requests, node:fs/promises handles files, node:child_process runs compilers and Git, node:path normalizes paths, node:crypto hashes authorization cache scopes, and node:os identifies the current machine. This keeps the backend predictable and package-light. The tradeoff is that routing and request handling are written manually. That is why handleApi is large: it plays the role that Express or another framework would normally play.

There are three important roots. root is the builder workspace that contains the site files and local draft catalog. portfolioRoot is the publish mirror, which may be separate from root so generated website files can be pushed without mixing in builder-only files. compileRoot is where project-local code workspaces and temporary compiler run folders live. Most path-related functions exist to make sure a request stays inside one of these roots.

The backend is also intentionally local-sensitive. Some endpoints are read-only, but write endpoints reject non-local callers. That is why functions such as `isLocalRequest` matter: the builder may be opened in a browser or accessed on a LAN during testing, but operations that change files, compile code, install tools, or publish should be performed only from the owner's computer.

Purpose Of The File

The purpose of `server.mjs` is to make the builder operational instead of decorative. Without it, `template-preview.html` and `template-preview.js` would still be able to render a screen, but they would not be able to reliably save drafts, upload project evidence, run compilers, authenticate GitHub targets, import an existing website, launch updates, or call private AI providers. The file therefore holds the application's real side effects.

The file is also a protection layer. It validates filenames, normalizes repository URLs, keeps publish authentication scoped to a machine and target, prevents stageable publishing paths from wandering outside the publish mirror, blocks remote write requests, caps text fetched from external sources, and strips terminal output into UI-safe strings. Those decisions are not cosmetic. They prevent the portfolio builder from becoming a general-purpose remote command runner or a tool that accidentally pushes the wrong files.

Primary Responsibilities

The first responsibility is serving local files. The same server that handles `/api` requests also serves `index.html`, `styles.css`, `script.js`, `template-preview.html`, project catalogs, assets, and uploaded docs during local preview. The static-serving logic checks that the requested path stays inside the workspace before reading the file. That simple check is what prevents a URL from escaping into arbitrary folders on the machine.

The second responsibility is compilation. Compile Code works because the backend can write source files to a project workspace, discover tools such as `node`, `python`, `gcc`, `g++`, `javac`, `iverilog`, `vvp`, and `LTspice`, run those tools with timeouts, collect stdout and stderr, cache successful builds, and return terminal output immediately to the frontend. The backend treats each language differently because JavaScript, Python, C, C++, Java, Verilog, SystemVerilog, LTspice, and HTML do not build the same way.

The third responsibility is publishing. Publishing is not just 'write `projects.json`.' The backend must verify that the publish mirror is a Git repository, know its origin remote and branch, check compatibility with a static portfolio site, synchronize with the remote to avoid non-fast-forward errors, stage only approved publish paths, commit if there are changes, and push to the selected branch. This is why the publishing code is careful and verbose.

The fourth responsibility is intelligence. For local AI, the backend can use an OpenAI API key or a local Ollama model. It also enriches portfolio context by reading public project links, same-site files, GitHub repositories, GitHub profiles, README files, and selected source files when the question calls for it. The browser cannot safely hold private API keys, so `server.mjs` keeps that path private.

The fifth responsibility is application maintenance. The update functions check GitHub Releases, download an installer, write a PowerShell handoff script, close the running app, run the installer, and relaunch the updated executable. This is backend work because the browser cannot replace the running desktop app.

Important Data Objects And Why They Matter

`compileLanguageProfiles` is the backend's language map. It tells the compiler system what default filename to use, which file extensions belong to each language, what human label to show, which tools are required, and which Winget package can install the tool if automatic installation exists. Without this object, `compileAndRunCode` would need hardcoded language assumptions scattered throughout the file.

`compileToolCandidates` is the tool-discovery map. It lists command names and likely Windows install paths for Git-adjacent and compiler-adjacent executables. `findTool` uses this object so the app can work even when a tool

is installed but not perfectly exposed on PATH. The caches `compileToolCache` and `compileToolVersionCache` exist so repeated compiles do not waste time rediscovering the same executables.

`publishPaths` is the publishing allowlist. It lists the site files and folders the backend is allowed to copy into the publish mirror and stage in Git. This is one of the most important safety objects in the file. Without an allowlist, an Apply to site operation could accidentally stage builder-only files, private drafts, temporary compile output, or unrelated local files.

`portfolioAiInstructions` is the system behavior contract for the portfolio assistant. It tells the model how to distinguish general conversation, general engineering, general knowledge, and portfolio-specific questions. It also tells the model when to use context, when not to invent, and how to handle public GitHub source excerpts. That string is long because it is the policy boundary for the AI feature.

Code Walkthrough

The walkthrough below groups related functions where they form one idea. Small helpers are explained together when that is clearer than pretending each one is a separate chapter. Larger functions receive direct explanations of their parameters, internal objects, side effects, return values, and why they are necessary.

Startup, roots, and response safety

These functions and constants establish the backend's working environment. They define where files live, how JSON responses are sent, and what security headers are attached.

securityHeaders(extra)

This function returns the baseline security headers added to local backend responses. The optional extra object lets a caller add or override headers without rewriting the common policy. Its purpose is to keep every response consistent: no MIME sniffing, strict referrer behavior, and browser permission limits. Without it, every endpoint would need to remember its own security headers, and missed endpoints could behave differently.

sendJson(response, status, data)

This is the standard JSON response writer. It receives the Node response object, an HTTP status code, and any serializable data object. It writes headers, disables caching, stringifies the data with indentation, and closes the response. The function matters because every API endpoint should return JSON in the same shape and with the same cache rules. It is intentionally synchronous around `response.end` because once the data object is ready, the output should be immediate.

createServer request handler

The final `createServer` block is the actual HTTP entry point. It first asks `handleApi` whether an `/api` request has been handled. If not, it treats the request as a static file request. The pathname is normalized, joined to root, and rejected if it escapes root. Then the file is read and sent with a MIME type from the types map. This is the simplest possible local server: API first, static files second, forbidden paths rejected, missing files return 404.

Language detection, filenames, and code formatting

These helpers make Compile Code feel automatic. They normalize language names, infer languages from file names and source text, produce safe filenames, and apply simple beautification before code is saved or displayed.

normalizeCodeLanguage(value), languageFromFileName(fileName, code), and detectCodeLanguageFromSource(code, fileName)

These functions form the language-detection chain. `normalizeCodeLanguage` turns aliases such as `js`, `py`, `cpp`, `c++`, `sv`, or `system verilog` into canonical ids. `languageFromFileName` prefers file extensions because `.java`, `.sv`, `.py`, and `.cpp` are strong evidence. `detectCodeLanguageFromSource` uses both filename and source clues, such as `public class` for Java or module syntax for HDL. The returned language id selects a compile profile, default filename, toolchain, highlighting rules, and build behavior.

safeCodeFileName(value, language)

This function accepts a requested filename and a language, then returns a filename safe enough to write under the compile workspace. It strips dangerous path characters, keeps a language-appropriate extension, and falls back to the default filename from `compileLanguageProfiles`. Without it, a user-provided filename could accidentally create confusing paths or miss the extension needed by a compiler.

beautifyCode(code, language) and indentBraceCode(code)

`beautifyCode` is a lightweight formatter used by the code editor. For brace-based languages it delegates to `indentBraceCode`, which adjusts indentation by watching braces and brackets. It is not a full compiler-grade

formatter, but it gives pasted code a cleaner shape. The function exists because code that looks organized is easier to read before being appended to a project.

Tool discovery and process execution

Compile Code, Git publishing, installers, and AI helpers all need controlled process execution. These functions find tools, run them safely, and return useful terminal text.

findExecutableUnder(folder, names, maxDepth) and findTool(toolName)

findExecutableUnder recursively searches likely install folders for executable names. findTool uses compileToolCandidates first, then command lookup, then known package folders. The important parameter is toolName: it is the key into the candidate map and cache. The function is async because filesystem searches and command lookup can take time. It stores successful results in compileToolCache so repeated compiler runs do not lag.

runProcess(command, args, options)

runProcess is the controlled command runner. command is the executable path or command name; args is the exact argument list; options can supply cwd, timeoutMs, env, and stdin input. The function spawns the process, collects stdout and stderr, kills the process on timeout, and resolves a structured object containing success state, output, exit code, elapsed time, and timeout status. Without this wrapper, every compiler and Git call would duplicate risky process code.

terminalLine, processTerminalText, replacePathReferences, and processTerminalTextWithPaths

These helpers turn raw process results into readable terminal output. They combine stdout and stderr, include elapsed time and status, and replace long local paths with shorter display names. They do not compile anything themselves; they make compiler and Git results understandable in the builder UI.

Compile workspace and build orchestration

This is the backend IDE layer. It turns editor state into real files, runs the correct language-specific toolchain, caches outputs, and returns terminal and waveform data.

saveCompileSource({ projectId, fileId, title, fileName, language, role, code, stdin })

saveCompileSource writes the active source file and a metadata file under compileRoot. The object parameter is the frontend's description of the active file. projectId chooses the project workspace, fileId chooses the file folder, fileName chooses the source filename, language chooses the compile profile, role distinguishes HDL design/testbench files, code is the source text, and stdin is optional runtime input. The returned saved object tells the frontend exactly where the backend stored the source.

compileWorkspaceFilesFromPayload(payload, activeFileName, activeLanguage)

This function normalizes the list of files sent by the frontend. It makes sure the active file is included, gives every file a safe id and filename, normalizes language and role, and removes duplicates. It matters because a project workspace is not just one file: C/C++ headers, Java classes, and HDL testbenches may all be needed together.

writeCompileWorkspaceSources(files, targetDir, options)

This async function writes a normalized list of workspace files into a run or cache directory. It can filter by languages or extensions, and it returns written source records that include sourcePath and uniqueName. Many compilers need real files on disk rather than strings in memory, so this function is the bridge between editor state and compiler input.

compileAndRunCode(payload)

compileAndRunCode is the central build engine. Its payload contains language, fileName, code, action, projectId, stdin, workspaceFiles, and forceRebuild. It detects the language, saves the active source, prepares run folders, selects the action, finds tools, runs language-specific compile/run/simulate logic, and returns ok,

language, saved metadata, terminal text, and waveform data when available. It is async because every meaningful operation may touch disk or external processes. Without it, the Compile Code UI would be only a text editor with no real execution path.

installCompilerTools(language)

installCompilerTools looks up the selected language profile and runs Winget packages when automatic installation is configured. Its input is the language id. Its output is a terminal-style result describing whether installation commands succeeded. It exists to reduce manual setup, but it intentionally returns a clear message for languages that do not have automatic installation configured.

C, C++, Java, and HDL build details

These helpers customize compileAndRunCode for languages that need multiple files, binary artifacts, cached builds, testbenches, or waveform output.

cFamilyCompileProfile, cFamilyWorkspaceSources, cFamilyBinaryName, and cFamilyRunOutput

These functions define how C and C++ behave. cFamilyCompileProfile chooses compiler flags, standard version, and whether a file is a header-only syntax check. cFamilyWorkspaceSources selects relevant .c/.cpp/.h/.hpp files. cFamilyBinaryName gives a safe executable name. cFamilyRunOutput formats runtime output. Together they let Compile Code distinguish syntax checking, building a workspace, and running a binary.

javaMainClassName(code, fileName)

Java execution needs a class name, not only a filename. This function reads the code to find a public class first, then any class, then falls back to the filename. compileAndRunCode uses that class name to know which .class file to expect and what to pass to the java runtime.

hdlFilesFromPayload, inferCompileFileRole, normalizeCompileFileRole, hdlModuleNames, and hdlHasWaveDump

These helpers make Verilog/SystemVerilog simulation sane. They identify design files versus testbenches, normalize the role labels, extract module names so a top module can be selected, and check whether a testbench contains \$dumpfile and \$dumpvars. Simulation needs stimulus and waveform dumping; these functions enforce that idea before vvp runs.

parseVcdScopeText(text, source), readHdlWaveform(cacheDir), and clearHdlWaveforms(cacheDir)

parseVcdScopeText converts a VCD waveform file into structured signal data: timescale, max time, signal names, and value changes. readHdlWaveform finds a generated VCD file in the cache directory and parses it. clearHdlWaveforms removes stale VCD files before a fresh simulation run. These functions are why the builder can show a signal scope instead of only text output.

GitHub, public source, and AI context enrichment

These functions let the assistant answer from project evidence, public GitHub links, public profile links, and fetched text without pretending to see private data.

sourceUrlAllowed, parseGitHubSourceUrl, fetchGitHubJson, and fetchLimitedText

These helpers control external reads. sourceUrlAllowed rejects unsafe or unsupported URLs. parseGitHubSourceUrl understands GitHub profile and repository URLs. fetchGitHubJson calls GitHub's public API with appropriate headers. fetchLimitedText retrieves bounded text so a large public file cannot flood the AI context. Together they keep enrichment useful and limited.

fetchGitHubProfileSource, fetchGitHubRepositorySource, and fetchGitHubSourceText

These functions are the GitHub-specific knowledge collectors. A profile source can yield profile metadata and repositories. A repository source can yield README text, repository metadata, and selected source files scored

against the question. The selected snippets become sourceExcerpts that the AI can quote or explain. They exist so a question about code can reach public GitHub evidence rather than only project summaries.

enrichPortfolioContext(context)

This function receives the context assembled by the frontend and expands it with fetched source excerpts when allowed. It reads sourceFetches, fetches safe public text, and returns a richer context object. The function is async because it may call GitHub, fetch same-site files, or read local files. Without it, the AI backend would only see the compact catalog context and would be weaker on uploaded files or public repositories.

Publishing target, authorization cache, and site push

This group is the site-deployment guardrail. It validates the target, verifies access, caches successful authorization, imports from target, and applies generated site files.

validatePublishRemoteUrl, parseGitHubRemote, validateCredentialPair, and validateCustomDomain

These functions validate user-provided publishing inputs. repository URLs must be plausible Git remotes, credentials must be paired when supplied, and custom domains must be simple valid domain names. They exist so configure/authenticate actions fail early with clear messages instead of half-writing a broken target.

resolveInsideRoot, resolveInsidePortfolioRoot, resolveInsideCompileRoot, and samePath

These path helpers defend the workspace boundaries. They turn user-controlled segments into paths inside the intended root and prevent path traversal. They are boring by design, but without them uploads, compile files, and publish sync could write outside the intended folders.

configurePublishTarget(options) and authenticateGitHubForTarget(options)

configurePublishTarget sets up the publish repository, remote, branch, optional custom domain, and stored credentials. authenticateGitHubForTarget goes further: it temporarily applies the target, checks Git availability, stores credentials if provided, synchronizes the remote branch, verifies write access, writes the auth cache, and only then keeps the target. This distinction matters: target setup should not be considered complete until write access is proven.

publishAuthCacheScope, writePublishAuthCache, publishAuthCachelsFresh, and assertPublishAccess

These functions control the 'do not ask every time' behavior. The cache stores remote, branch, repository, machine/user scope, checked time, expiration, and extended trust history. assertPublishAccess first validates repository shape, then reuses a fresh cache if the exact target and machine scope match; otherwise it performs a new write check. Without this group, the app would either annoy the owner constantly or trust stale authorization too broadly.

syncFromPublishTarget()

syncFromPublishTarget authenticates, clones the selected branch into a temporary folder, checks for a compatible projects.json, backs up local compatible files, copies imported site assets into the builder workspace, updates the local draft, and syncs publish files. This is the 'load from target' behavior for moving to another machine or restoring from the live repository.

syncPortfolioPublishFiles(options) and publishSiteChanges(publishAccess)

syncPortfolioPublishFiles copies only allowed publishPaths from the builder workspace to the publish mirror. publishSiteChanges then synchronizes the branch, bumps asset versions, stages allowed paths, commits changes when needed, and pushes. This pair turns local saved portfolio output into the public website while keeping builder-only files out of the publish target.

Updates, security reporting, AI, and API routing

These functions make the backend responsible for application maintenance and the public assistant endpoint used during local preview.

getUpdateInfo(), getBuilderReleaseDownloadReport(), safeUpdateFileSegment(), powershellSingleQuoted(), and downloadAndLaunchAppUpdate()

getUpdateInfo reads release information and decides whether a newer builder exists.

downloadAndLaunchAppUpdate downloads the latest installer, writes a PowerShell handoff script, closes running builder processes, runs the installer, and relaunches the app. The helper functions sanitize version/file fragments and PowerShell strings so generated scripts are safer. This group exists because a running desktop app cannot replace itself like a normal webpage refresh.

getSecurityReport()

getSecurityReport summarizes what the builder can and cannot know about visitors, downloads, public site headers, local-only protections, and authentication caching. It is intentionally honest: a static GitHub Pages site cannot identify every visitor without an analytics backend.

callOllamaPortfolioAi, extractOpenAiText, extractOllamaText, ruleBasedConversationAnswer, and handlePortfolioAi

These functions make the local AI endpoint. handlePortfolioAi reads the question, conversation, intent, and context; enriches the context; chooses OpenAI if an API key exists; otherwise tries Ollama; otherwise returns a clear unavailable response. The extractor helpers normalize provider response shapes.

ruleBasedConversationAnswer covers simple local fallback conversation. This keeps AI secrets on the backend side and gives the frontend one endpoint to call.

handleApi(request, response, url)

handleApi is the router for every backend endpoint. It handles read endpoints such as catalog, templates, system check, update info, security report, and compiler tools. It rejects non-local write requests, then handles write operations such as AI, code beautify, code save, code compile, tool install, Git authentication, load from target, save draft, apply to site, and upload. It returns true when it handled a route so the static file server does not also try to serve it.